Beadandó feladat dokumentáció – Róka–nyúl szimuláció

Oct 8, 2026

Készítette: Hevesi Georgina, Y2V0WV · e-mail: hvsbc@student.elte.hu

Feladat

Készítsünk programot, amellyel a következő szimulációt futtathatjuk.

Adott egy *n*×*n* elemből álló játékpálya, amelyen nyulak és rókák élnek. A pálya mezőin fű nőhet. Az állatok adott időközönként lépnek egy mezőt (vízszintesen, függőlegesen vagy átlósan), és minden lépéssel energiát veszítenek.

A nyulak fűvel táplálkoznak: ha fűvel borított mezőre lépnek, megeszik a füvet, és energiát nyernek. A rókák a nyulakat vadásszák: ha a szomszédos mezők egyikén nyulat látnak, arra lépnek, megeszik, és energiát nyernek. Ha egy állat energiája elfogy, elpusztul. Ha egy állat energiája meghaladja a faja szaporodási küszöbét, a szabad szomszédos mezők egyikén utódot hoz létre, és az energiája megoszlik közte és az utódja között. A megevett fű adott eséllyel nő vissza.

A pálya méretét, a kezdeti nyulak és rókák számát, valamint a szabályok paramétereit (a fű visszanövésének esélye, a szaporodási küszöbök, az étel energiája, a lépésenkénti energiaveszteség, a lépések közötti idő) új játék indításakor adjuk meg. A játék állapota fájlba menthető, és onnan visszatölthető.

A program biztosítson lehetőséget új játék kezdésére, a szimuláció elindítására, szüneteltetésére (ekkor nem telik az idő), valamint léptetésére egyesével. Az állatok szerkeszthetők: új nyulat vagy rókát húzással lehet a pályára tenni, a meglévőket pedig el lehet távolítani, a szerkesztés idejére a szimuláció automatikusan szünetel. Ismerje fel, ha vége a játéknak (valamelyik faj kihalt), és jelezze. A program játék közben folyamatosan jelezze ki az eltelt lépések számát, a nyulak és a rókák számát, valamint ábrázolja a két populáció alakulását.

Elemzés

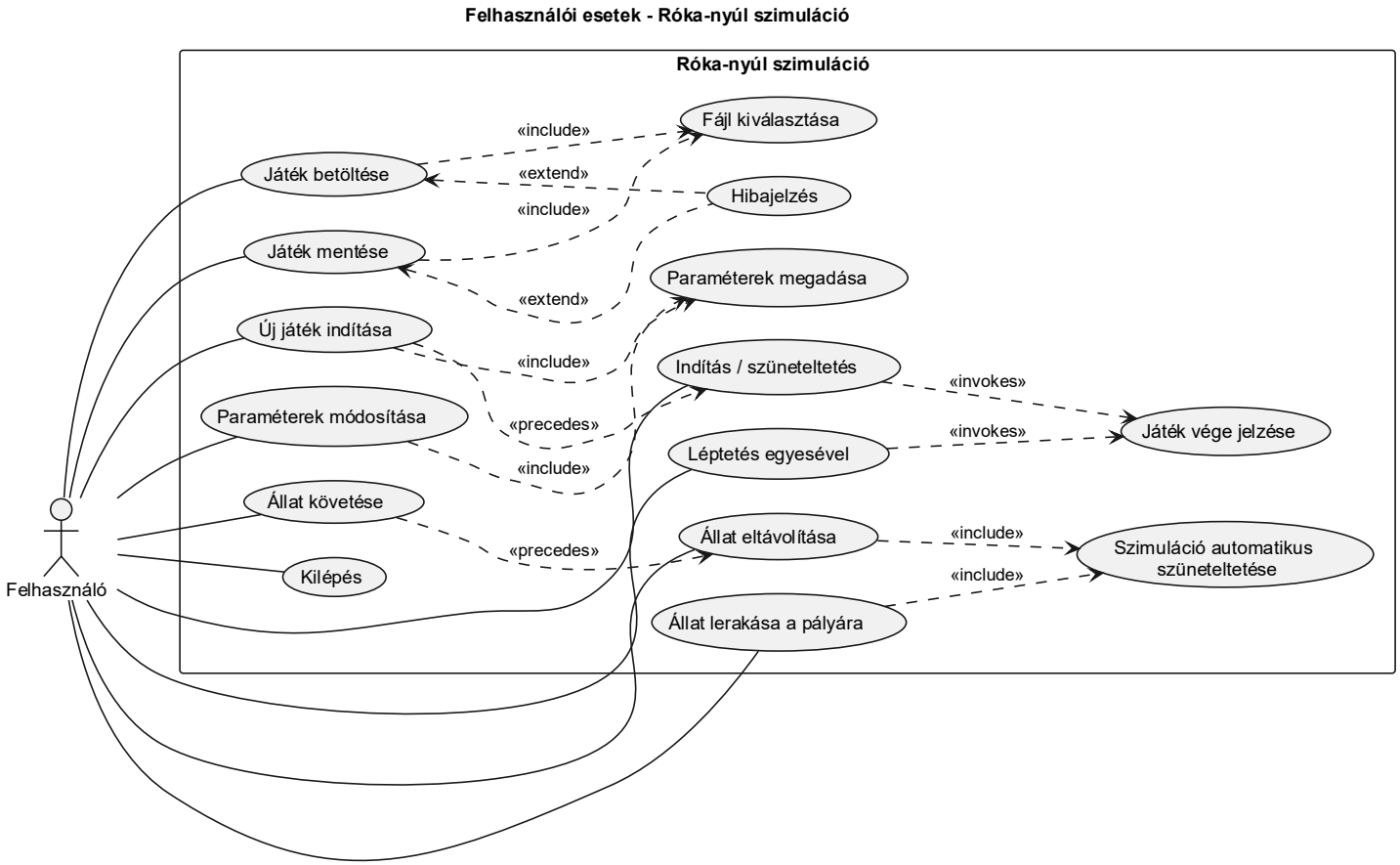
- A játékban három pályaméret kínálunk: a 10×10-es, a 20×20-as és a 30×30-as. A játék állapotát egyszerű szöveges fájlban (.sim) tároljuk, az alkalmazás ilyen fájlokat tölt be és ment.
- A feladatot egyablakos asztali alkalmazásként Windows Forms grafikus felülettel valósítjuk meg.
- Az ablakban elhelyezünk egy menüt a következő menüpontokkal: **Game** (New game, Change parameters, Exit), **Start / Pause / Resume**, **Next step**, **File** (Load, Save). Az ablak alján megjelenítünk egy státuszsort, amely az eltelt lépések számát, valamint a nyulak és a rókák számát jelzi.
- A játéktáblát egy n×n-es mezőkből álló rács reprezentálja, amelyen látszik a fű, az állatok (animált ikonnal) és az energiájuk (energiacsík). Egy állatra kattintva az oldalsó panelen egy kártya követi az energiáját, onnan el is távolítható.
- Az oldalsó panelen állatpaletta található: egy nyulat vagy egy rókát a pályára húzva új állatot helyezünk el. A szerkesztés (új állat, állat törlése) idejére a szimuláció automatikusan szünetel, majd – ha futott – folytatódik; a léptetés is csak szüneteltetett játékban lehetséges. A két faj létszámának alakulását egy diagram mutatja.
- A játék automatikusan feldob egy dialógusablakot, amikor vége a játéknak (valamelyik faj kihalt). Szintén dialógusablakokkal végezzük el az új játék és a paraméterek megadását, a betöltést és a mentést, valamint ezek sikertelenségének jelzését.

A felhasználói esetek az alábbi ábrán láthatóak:

felhasználói esetek · 10 funkció, 1 szereplő

A kiemelt esetek az állatok szerkesztését jelentik: ezek idejére a szimuláció automatikusan szünetel.

Tervezés



Programszerkezet

A programot háromrétegű architektúrában valósítjuk meg. A megjelenítés a View, a modell a Model, míg a perzisztencia a Persistence névtérben helyezkedik el. A rétegek függősége egyirányú: a View csak a Model-t ismeri, a Model a Persistence-t, a Persistence pedig egyik másikról sem tud.

A program szerkezetét két projektre osztjuk implementációs megfontolásból: a Persistence és Model csomagok a program felületfüggetlen projektjében (FoxBunnySimCore), míg a View csomag a Windows Formstól függő projektjében (FoxBunnySimView) kap helyet.

Perzisztencia

Az adatkezelés feladata a szimuláció állapotával kapcsolatos információk tárolása, valamint a betöltés és a mentés biztosítása.

A GameState rekord egy teljes játékállást ír le egyszerű adatokkal: a tábla méretét, a fűtérképet (a mezőkre ismert, hogy van-e rajtuk fű), az állatokat (AnimalData), a populáció történetét (PopulationData), az eltelt lépések számát és a szabályokat (RulesData). A perzisztencia a saját adattípusait használja, így nem ismeri a modell típusait.

Az IDataAccess interfész lehetőséget ad az állás betöltésére (LoadAsync) és mentésére (SaveAsync), ezeket hatékonysági okokból aszinkron módon valósítjuk meg.

Az interfészt szöveges fájl alapú adatkezelésre a FileDataAccess osztály valósítja meg, amely a SimFileReader és a SimFileWriter osztályokat használja. A fájlkezelés során fellépő hibákat a SimFileException kivétel és leszármazottai jelzik (GameFileAccessException, GameFileEndedEarlyException, GameFileSyntaxException, GameFileValueException), amelyek megadják a hibás sor számát is.

A .sim fájl első sora megadja a tábla méretét, az állatok számát, az eltelt lépések számát és a szabályok értékeit. A fájl többi része az alábbi: előbb az állatok sorai (a faj 0 vagy 1 lehet: 0 a nyúl, 1 a róka), majd a tábla *n* hosszú sorai (0: nincs fű, 1: van fű), végül a populáció története.

```
n m t interval grassGrowth bunnyReprod foxReprod foodEnergy energyLoss
x y energy species        (m sor, species: 0 = nyúl, 1 = róka)
0 1 1 0 ...               (n sor, soronként n darab 0/1)
h                         (a történet hossza)
bunnies foxes             (h sor)
```

Modell

A modell lényegi részét a GameModel osztály valósítja meg, amely szabályozza a tábla tevékenységeit, valamint a játék egyéb paramétereit. A típus lehetőséget ad a játék folytatására, megállítására (ResumeGame, PauseGame), valamint lépésre (Step). Új játéknál a megadott paraméterek alapján (SimulationParameters) véletlenül generálódnak a kiinduló mezők (CreateNew), egy mentett játékot a LoadAsync tölt be, a SaveAsync pedig elment.

- A játék időbeli kezelését egy időzítő végzi (_timer, a TimerAggregation osztály, amely az ITimer interfészből származik), amelyet inaktiválunk (PauseGame), amíg az állatokat szerkesztjük, majd újraindítjuk (ResumeGame).
- A játék állapotváltozásáról a GameAdvanced, a játék végéről a GameOver esemény tájékoztat.
- Az állatokat az Animal absztrakt osztály írja le, amelynek leszármazottai a Bunny és a Fox (célmező választása: ChooseTarget, a mező elfogyasztása: CanEat). Egy körök – evés, szaporodás, túlélés ellenőrzése, mozgás – az Animal.Move metódus feladata. A fajt a Species felsorolás írja le.
- A modell nem ad át élő állatobjektumot a nézetnek: az állatokról egy AnimalStatus pillanatképet ad (azonosító, pozíció, faj, energia, szaporodási küszöb), és az állatokat azonosító (Id) alapján kérdezhetjük le és törölhetjük (GetStatus, FindAnimal, RemoveAnimal).
- A perzisztencia hibáit a GameFileException csomagolja be, így a nézet nem ismeri a perzisztencia típusait. A fájlban használt fajszámok (0, 1) és a Species értékek közötti leképezést is a modell végzi.

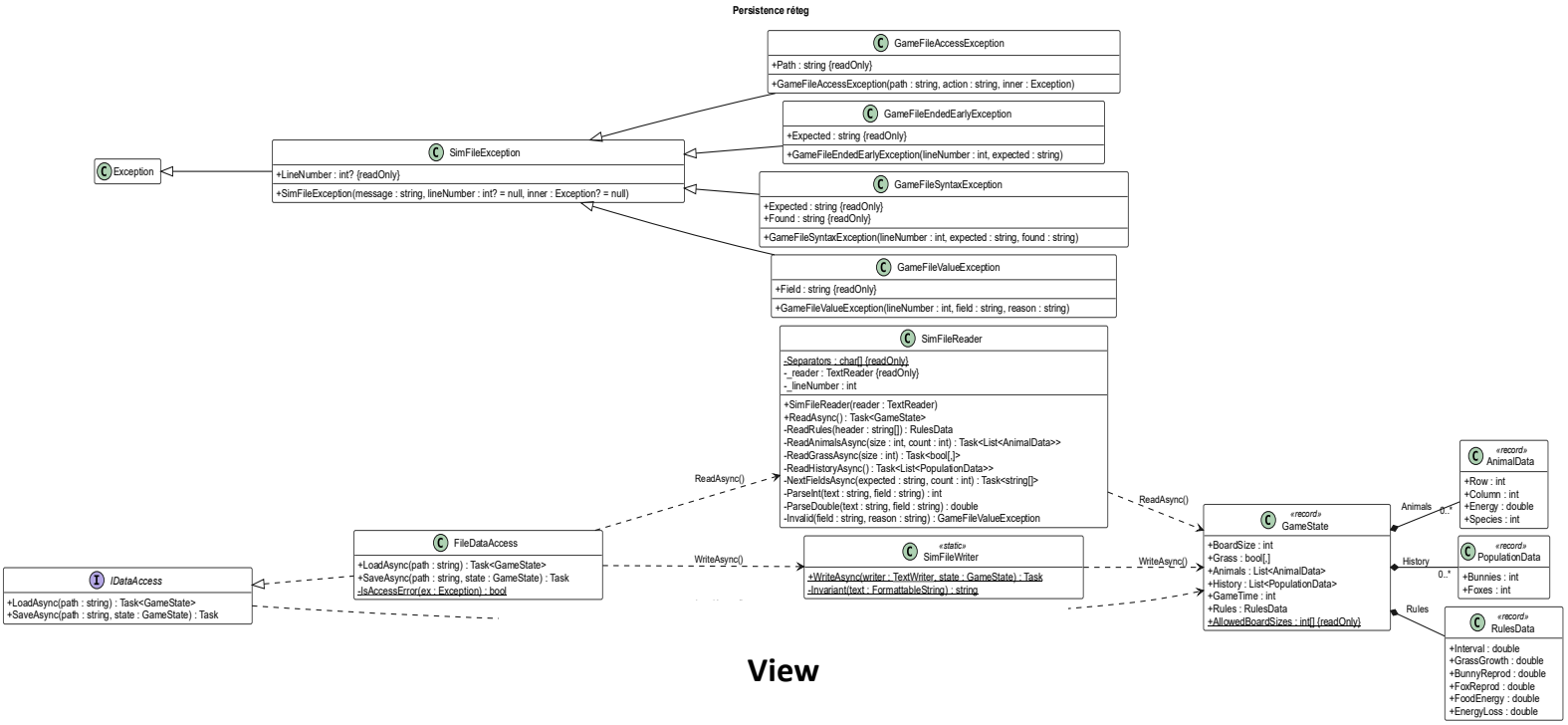
Nézet

A nézetet a MainForm osztály biztosítja, amely tárolja a modell egy példányát (_model). A nézet csak a megjelenítéssel és a felhasználói bemenettel foglalkozik: az adatokat mindig a modelltől kéri el, és nem tartja a saját másolatát.

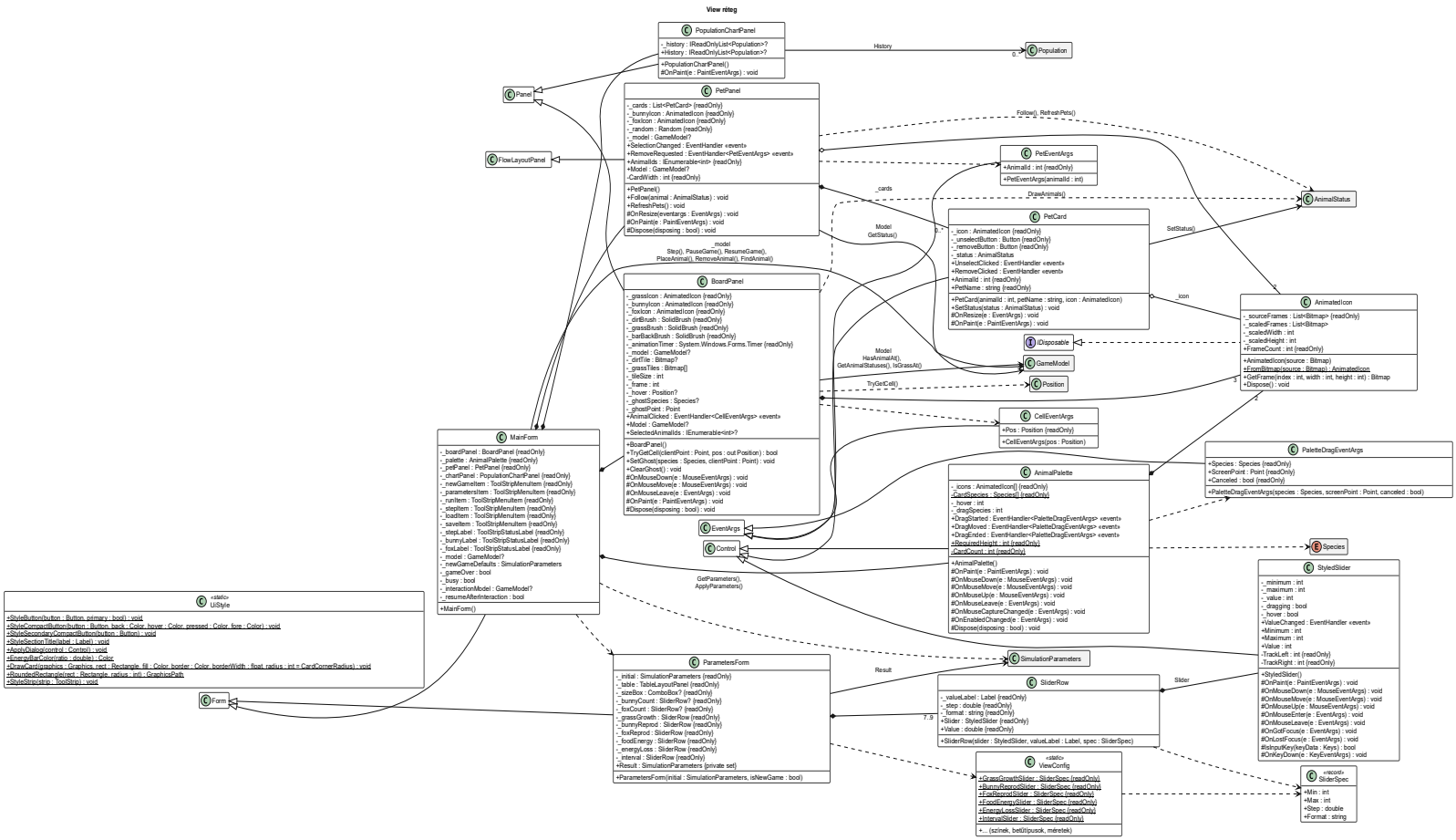
- A játéktáblát a BoardPanel egyedi vezérlő rajzolja (fű, állatok animált ikonja, energiacsíkok, kiemelések). Az állatpaletta (AnimalPalette) húzással teszi lehetővé új állatok lerakását, a PetPanel kártyái (PetCard) a kiválasztott állatokat követik, a PopulationChartPanel pedig a két faj létszámának alakulását rajzolja.
- A felületen létrehozzuk a megfelelő menüpontokat, státuszsort, valamint a dialógusablakokat és a hozzájuk tartozó eseménykezelőket. Az új játék és a paraméterek megadását a ParametersForm (StyledSlider csúszkákkal) végzi. A színek, méretek és a stílus a ViewConfig és a UiStyle osztályokban van egy helyen.
- Mivel az időző másik szálon fut, a modell eseményeire a nézet a felületi szálra átváltva (BeginInvoke) reagál.

Az osztálydiagram ezeken az ábrákon látható:

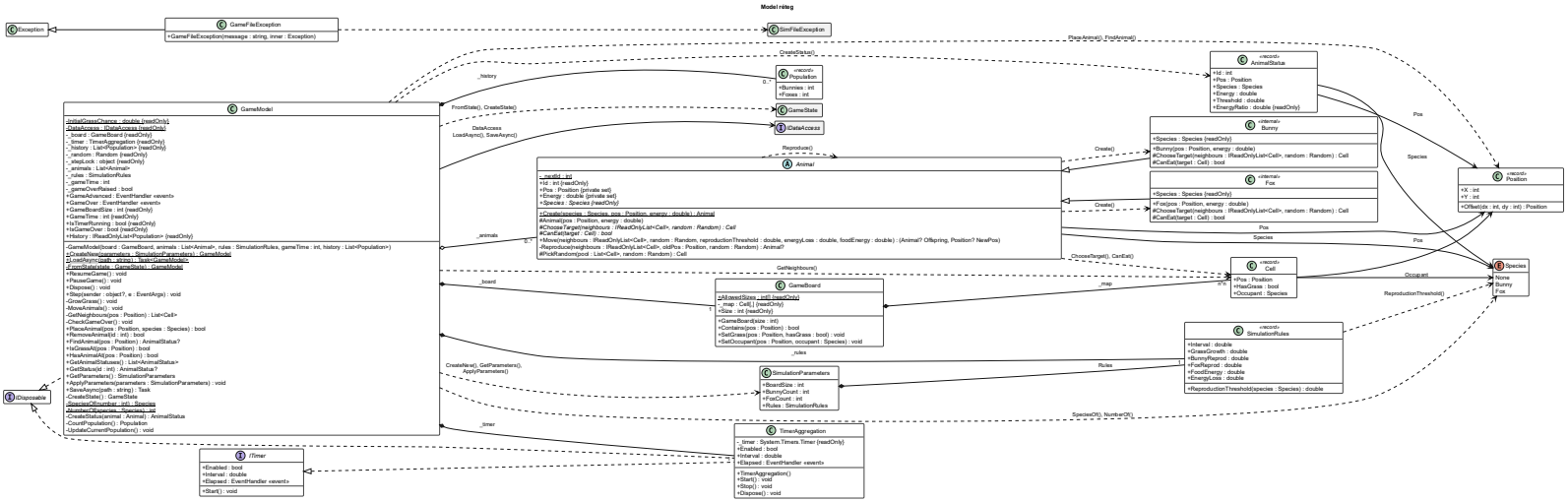
Perzisztencia



View



Model



Tesztelés

A modell funkcionalitása egységtesztek segítségével lett ellenőrizve a GameModelTest osztályban. A tesztek a modell nyilvános Step metódusát közvetlenül hívják, így az időzítőt nem kell kivárniuk. A betöltést és a mentést a tesztek ideiglenes .sim fájlokkal ellenőrzik, amelyeket az eredeti SimFileWriter, illetve SimFileReader osztályok írnak és olvasnak, a fájlokat pedig a tesztek végén töröljük.

Az alábbi tesztek kerültek megvalósításra:

- Új játék felállítása megfelelő-e (GameModelNewGameTest)
- Betöltés: a fájlban lévő fajszámok (0 nyúl, 1 róka) a megfelelő fajt adják-e (GameModelLoadTest)
- Sikertelen betöltés lekezelése (GameModelLoadErrorTest)
- Mentés: a nyúl 0-ként, a róka 1-ként kerül a fájlba (GameModelSaveTest)
- Egy lépés: idő, előzmény, esemény és energiaveszteség (GameModelStepTest)
- Evés: a nyúl megeszi a fűvet, a róka megeszi a szomszédos nyúlat (GameModelEatingTest)
- Szaporodás a küszöb felett (GameModelReproductionTest)
- Játék vége feltételek teljesülése és lekezelése (GameModelGameOverTest)
- Állatok szerkesztése: lerakás, törlés, és a szerkesztés tiltása futó játéknál (GameModelEditAnimalsTest)